

3. Lenguajes de programación y paradigmas

Clasificación · Tipado · Paradigmas · Ecosistemas y estándares · Elección del lenguaje

3.1 Criterios de clasificación — nivel de abstracción

El nivel de abstracción indica qué tan lejos está el lenguaje del hardware. Analogía: programar en ensamblador es construir una silla especificando cada clavo y cada corte; programar en Python es pedirle la silla a un carpintero — tú describes lo que quieres y él resuelve los detalles.

Nivel	Cómo trabaja	Ejemplos	Cuándo se usa	Limitación
Bajo nivel	Directamente con registros y memoria del procesador. Control total y rendimiento máximo, pero exige conocer el hardware.	Ensamblador (cada instrucción ≈ una operación de la CPU)	Drivers, firmware de microcontroladores, partes críticas de SO	Muy difícil de escribir, leer y mantener; un programa simple ocupa cientos de líneas
Alto nivel	Abstrae el hardware: escribes lógica y el compilador/intérprete la convierte en instrucciones. Código legible y portable entre sistemas.	Python, Java, JavaScript, C#, Kotlin	Web, apps móviles, análisis de datos — casi todo el software de usuario	Cedes algo de control y rendimiento (irrelevante para la mayoría de proyectos)

¿Y C? Ocupa una posición intermedia: más legible que el ensamblador, pero con acceso directo a memoria y punteros. Por eso domina en sistemas operativos (Linux está escrito en C) y en dispositivos con pocos recursos.

3.1 Criterios de clasificación — propósito y dominio

Tipo	Descripción	Ejemplos	Cuándo usarlo
Propósito general	Sirven para casi cualquier tipo de programa	Java, Python, C++	Cuando el proyecto no encaja en un dominio específico
Dominio específico (DSL)	Diseñados para un problema concreto; no tienen sentido fuera de ese contexto	SQL (bases de datos), HTML/CSS (web), MATLAB (cálculo numérico)	Cuando el problema encaja exactamente en su área

Áreas donde destaca cada lenguaje de propósito general		
Área	Lenguajes habituales	Por qué
Sistemas operativos y drivers	C, C++, Rust	Acceso directo al hardware, sin VM
Aplicaciones web	JavaScript, PHP, Python, Java	Grandes ecosistemas de frameworks web
Datos e inteligencia artificial	Python, R, Julia	Librerías especializadas (NumPy, pandas, TensorFlow)
Aplicaciones móviles	Kotlin (Android), Swift (iOS)	Integración nativa con cada plataforma
Dispositivos embebidos e IoT	C, Rust	Bajo consumo de memoria y energía

3.2 Tipado — cuándo se comprueban los tipos

El tipo de dato define qué clase de valor almacena una variable y qué operaciones admite. Los lenguajes difieren en CUÁNDO comprueban los tipos — y eso afecta a cuántos errores detectas en compilación y cuántos llegan a producción.

	Tipado estático	Tipado dinámico
Cuándo se comprueba	Antes de ejecutar: el compilador revisa los tipos y no deja compilar si hay error	Mientras el programa se ejecuta
Ejemplo	Java: <code>int resultado = edad + nombre;</code> → error en compilación: tipos incompatibles	Python: <code>x = 5</code> y luego <code>x = "hola"</code> — Python no protesta
Consecuencia	El error aparece mientras desarrollas, no en manos del usuario	Código más corto y flexible, pero los errores de tipo pueden aparecer en producción
Lenguajes	Java, C, C++, Rust, Kotlin, TypeScript	Python, JavaScript, Ruby, PHP

Inferencia de tipos: lenguajes estáticos que deducen el tipo según el valor asignado. Kotlin: `val edad = 25` (sabe que es Int), `val nombre = "Ana"` (String). Código más limpio que Java clásico con las mismas garantías en compilación.

3.2 Tipado — cuánta libertad dan con los tipos

Estático/dinámico describe CUÁNDO se comprueban los tipos. Fuerte/débil describe CUÁNTO permite el lenguaje mezclarlos:

	Fuerte	Débil
Qué hace	No mezcla tipos sin conversión explícita	Convierte tipos automáticamente
Ventaja	Menos errores silenciosos	Código más corto
Riesgo	Más verboso al escribir	Resultados inesperados difíciles de detectar
Ejemplo	Python, Java	JavaScript, PHP

```
# Python - fuerte
resultado = "3" + 2      # TypeError
                           inmediato
resultado = int("3") + 2  # hay que ser
                           explícito → 5
```

```
// JavaScript - débil
"3" + 2    // "32"   convierte el 2 a string
"3" - 2    // 1      aquí sí convierte "3" a
número
true + 1   // 2      true se convierte en 1
```

Débil no significa malo, pero exige más cuidado. JavaScript priorizó la facilidad de uso en el navegador en sus primeros años; TypeScript nació precisamente para añadirle tipado estático encima.

3.3 Paradigmas de programación (I) — imperativo y POO

Un paradigma es una forma de pensar y organizar el código: un estilo, no una característica técnica. La mayoría de lenguajes modernos soportan varios a la vez (Java añadió elementos funcionales; Python es multiparadigma desde el principio).

Imperativo / Procedimental — describes CÓMO, paso a paso

```
int suma = 0;
for (int i = 1; i <= 10; i++) { suma += i; }    // acumula
en cada iteración
printf("Suma: %d\n", suma);
```

El estilo más cercano a la CPU. Para algoritmos directos y lógica donde el orden importa. Lenguajes: C, Pascal — y presente en casi todos.

Orientado a objetos (POO) — clases (moldes) y objetos (instancias) con estado propio

```
class Coche {
    String marca;  int velocidad;
    void acelerar(int inc) { velocidad += inc; }    //
modifica su propio estado
}
Coche miCoche = new Coche();
miCoche.acelerar(50);    // solo afecta a miCoche, no a
otros Coche
```

Para modelar entidades con estado propio (usuarios, productos, vehículos). El paradigma dominante en software empresarial. Lenguajes: Java, C++, C#, Kotlin, Python.

3.3 Paradigmas de programación (II) — funcional y lógico

Funcional — describes QUÉ quieres obtener, no cómo

```
// Imperativo: el cómo
List<Integer> pares = new ArrayList<>();
for (int n : numeros) { if (n % 2 == 0) pares.add(n); }

// Funcional (Java streams): el qué
List<Integer> pares = numeros.stream()
    .filter(n -> n % 2 == 0)
    .collect(Collectors.toList());
```

Funciones puras: mismo input → mismo output, sin modificar datos externos. Para transformación de datos y procesamiento paralelo; código más predecible.

Lenguajes: Haskell, Scala, Elixir — y streams en Java, map/filter/reduce en JS y Python.

Lógico — describes HECHOS y REGLAS; el motor deduce la respuesta

```
humano(socrates).           % hecho
humano(platon).             % hecho
mortal(X) :- humano(X).     % regla: todo humano es mortal
?- mortal(socrates).        % → true (sin bucles ni
condiciones)
```

Para sistemas expertos, IA clásica, problemas con restricciones (horarios, configuraciones). Lenguaje: Prolog. El menos común, pero conceptualmente muy distinto.

3.3 Paradigmas de programación (III) — eventos y reactivo

Orientado a eventos — el programa espera; cuando ocurre algo, ejecuta el manejador

```
// Este código no hace nada hasta que el usuario haga clic
document.getElementById("boton").addEventListener("click",
function() {
    alert("¡Botón pulsado!");
});
```

El programa no se ejecuta de arriba abajo: registra manejadores y queda a la espera (clic, mensaje, respuesta de red). Para interfaces gráficas y apps web. JavaScript en el navegador es el ejemplo más claro.

Reactivo — flujos de datos: te suscribes y recibes actualizaciones automáticas

```
// Cada vez que el usuario escribe en el buscador,
// se lanza una búsqueda automáticamente
busqueda$.subscribe(termino => {
    buscarResultados(termino); // se ejecuta solo cuando
    cambia el valor
});
```

Analogía: una hoja de cálculo — cambias una celda y las que dependen de ella se actualizan solas. Para datos en tiempo real: chats, monitorización, resultados en vivo. Es un estilo, no un lenguaje: lo verás en frameworks frontend de DAW/DAM.

Resumen: imperativo = el CÓMO · POO = entidades con estado · funcional = el QUÉ · lógico = hechos y reglas · eventos = reaccionar a acciones · reactivo = suscribirse a cambios.

3.4 Ecosistemas y estándares

Muchos lenguajes evolucionan a través de organismos y procesos formales que garantizan que distintos compiladores e intérpretes se comporten igual. Sin estándar para C, el código compilado con gcc podría no funcionar con clang.

Lenguaje	Organismo / proceso	Qué regula
C / C++	ISO (International Organization for Standardization)	El comportamiento de los compiladores. El estándar más usado hoy es C++20.
JavaScript	ECMA International → especificación ECMAScript (ES)	Sintaxis y comportamiento del lenguaje. ECMAScript es el nombre formal del estándar; cada año sale versión nueva (ES2015, ES2022...).
Python	PEPs (Python Enhancement Proposals)	Propuestas públicas de mejora: cualquiera puede proponer, la comunidad debate y un comité decide. Así entraron los f-strings o el operador :=.
Java	JSRs (Java Specification Requests) vía el JCP (Java Community Process)	Cómo evoluciona la plataforma Java. Empresas y desarrolladores participan votando y revisando propuestas.

3.5 Elección del lenguaje según el proyecto

Elegir lenguaje no es cuestión de gustos: depende del entorno de ejecución, del rendimiento necesario, de las librerías disponibles y de lo que sabe el equipo. Muchos proyectos mezclan varios: backend en Java, frontend en JavaScript, scripts en Python.

Escenario	Lenguajes habituales	Razón principal
Sistemas operativos y drivers	C, C++, Rust	Control directo del hardware, sin capa intermedia
Web — frontend (navegador)	JavaScript, TypeScript	Es el único lenguaje que los navegadores ejecutan de forma nativa
Web — backend (servidor)	Python, Java, Node.js, PHP	Grandes ecosistemas de frameworks web
Aplicaciones móviles	Kotlin (Android), Swift (iOS)	Integración nativa con el SDK de cada plataforma
Datos e inteligencia artificial	Python, R	Librerías maduras para análisis y aprendizaje automático
Embebidos e IoT	C, Rust	Bajo consumo de memoria, sin necesidad de VM

3.6 Perfil de los lenguajes más comunes

Referencia rápida de los lenguajes que más aparecerán en el módulo (no es una clasificación definitiva: los lenguajes evolucionan y amplían su ámbito):

Lenguaje	Nivel	Tipado	Paradigma principal	Uso típico
C	Medio-bajo	Estático, fuerte	Procedimental	SO, drivers, firmware, embebidos
Java	Alto	Estático, fuerte	Orientado a objetos	Apps empresariales, Android, backend
Python	Alto	Dinámico, fuerte	Multiparadigma	Datos, IA, scripting, web
JavaScript	Alto	Dinámico, débil	Eventos, funcional	Frontend web, backend (Node.js)
Kotlin	Alto	Estático, con inferencia	OO + funcional	Android, apps multiplataforma
Rust	Medio-alto	Estático, fuerte	Multiparadigma	Sistemas de alto rendimiento, embebidos